"Smart Event Management System"

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

CHERUKUMALLI SAI HIMA SEKHAR : 2520030554
RAMGARI RAGHUVEER : 2520030225
PENDRI SAIVARSHITH REDDY : 2520030598

Team Number: 14
Section: 2

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Smart Event Management System"** is a record of Bonafide work of **Cherukumalli Sai Hima Sekhar – 2520030554, Ramgari Raghuveer – 2520030225, Pendri Saivarshith Reddy – 2520030598** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Cherukumalli Sai Hima Sekhar – 2520030554
Ramgari Raghuveer – 2520030225
Pendri Saivarshith Reddy – 2520030598

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Smart Event Management System"** is a bonafide work done and submitted by **CHERUKUMALLI SAI HIMA SEKHAR – 2520030554, RAMGARI RAGHUVEER – 2520030225, PENDRI SAIVARSHITH REDDY – 2520030598** in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his continuous guidance, valuable suggestions, and constant encouragement throughout the development of this project. His expertise in Data Structures and Algorithms and his constructive feedback helped us shape the system architecture, select appropriate algorithms, and refine the implementation.

We are equally thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the necessary academic environment, facilities, and support that enabled us to complete this project successfully. His leadership and commitment to academic excellence have been a source of motivation.

We also extend our thanks to the faculty members of the Department of Computer Science and Engineering and to our classmates for their helpful discussions and support during the course of this work. Finally, we are grateful to our families for their patience and encouragement.

Cherukumalli Sai Hima Sekhar – 2520030554

Ramgari Raghuveer – 2520030225

Pendri Saivarshith Reddy – 2520030598

Abstract

The Event Management System is a terminal-based Java application designed to efficiently manage, organize, search, and optimize college and organizational events. The system stores event details such as title, date, venue, category, capacity, and participants, allowing users to search, retrieve, and explore events through an interactive command-line interface.

Advanced algorithms from the course outcomes are integrated into the system. String-matching algorithms including the Z-Algorithm are used for exact event and keyword searching with linear-time complexity. When exact matches are not found, Edit Distance (Levenshtein Distance) computed via Dynamic Programming provides fuzzy, typo-tolerant suggestions. Multiple Dynamic Programming paradigms—Interval DP, Bitmask DP, Tree DP, and Subset DP (Knapsack-style)—are applied to event scheduling optimization, venue allocation, and resource coverage problems.

The system demonstrates practical applications of advanced Data Structures and Algorithms by mapping theoretical concepts to real-world event-management scenarios. It runs entirely through a command-line interface using Java and file-based storage, making it lightweight, portable, and suitable for academic demonstration and further extension.

Table of Contents

Acknowledgementi
Abstractii
Table of Contentsiii
List of Figuresiv

1. Introduction1
- 1.1 Overview
- 1.2 Problem Statement
- 1.3 Scope of the Project
- 1.4 Objectives

2. System Requirements5
- 2.1 Functional Requirements
- 2.2 Non-Functional Requirements

3. System Design8
- 3.1 High-level Architecture
- 3.2 Description of each Component
- 3.3 Algorithm Selection Rationale
- 3.4 Search Flow Diagram

4. Implementation13
- 4.1 Module-wise Implementation
- 4.2 String Matching Module (Z-Algorithm)
- 4.3 Fuzzy Search Module (Edit Distance DP)
- 4.4 Dynamic Programming Modules
- 4.5 Main Control Flow and User Interface

5. Results24

6. Challenges & Limitations27
- 6.1 Issues Faced During Development
- 6.2 Solutions Applied
- 6.3 Limitations of the System

7. Conclusion & Future Work30

7.1 Conclusion

7.2 Future Work

7.3 Skills Learned During Development

8. References33

9. List of Figures

Fig. 3.1 High-level System Architecture

Fig. 3.2 Search Flow Diagram (Exact + Fuzzy)

Fig. 3.3 Module Interaction Diagram

Fig. 4.1 Z-Algorithm Working Illustration

Fig. 4.2 Edit Distance DP Table Example

Fig. 4.3 Interval DP Recurrence Structure

Fig. 5.1 Sample Console Output – Exact Match

Fig. 5.2 Sample Console Output – Fuzzy Suggestion

1. INTRODUCTION

1.1 Overview

Colleges and organizations regularly host a large number of events such as workshops, seminars, cultural festivals, technical symposiums, and competitions. Managing these events efficiently—searching for relevant events, avoiding schedule conflicts, allocating venues and volunteers, and providing a smooth experience for both organizers and participants—is a non-trivial task when done manually or with rudimentary tools.

This project presents a Smart Event Management System implemented as a terminal-based Java application. The system focuses on two core capabilities: (1) fast and intelligent event search, and (2) demonstration of advanced Dynamic Programming techniques applied to event scheduling and resource optimization problems.

Event data is stored as plain-text files in a data directory. Users interact with the system through a command-line interface. When a user enters a search query, the system first attempts an exact match using the Z-Algorithm (linear-time string matching). If no exact match is found, it falls back to a fuzzy search based on Edit Distance computed via Dynamic Programming, suggesting the closest event names (handling typos and partial matches). After every search, the system also displays illustrative results from multiple DP algorithms that model real scheduling and allocation scenarios.

The project is designed to map directly to the course outcomes of Data Structures and Algorithms – 3, covering string algorithms, multiple DP paradigms, and complexity analysis in a practical setting.

1.2 Problem Statement

Manual event management and simple keyword-based tools suffer from several problems:

- Slow searching when the number of events grows large.
- Inability to tolerate typos or approximate queries, leading to failed searches.
- Difficulty in detecting and resolving scheduling conflicts across venues and time slots.
- Inefficient allocation of limited resources such as seats and volunteers.
- Lack of a unified, algorithmically grounded demonstration of advanced DSA techniques applied to a realistic domain.

There is a need for a system that combines efficient string matching, typo-tolerant search, and optimization algorithms to address these issues while remaining lightweight and educational.

1.3 Scope of the Project

The scope of the present work includes:

- Loading event information from text files stored in a local data directory.
- Exact keyword search across event content using the Z-Algorithm.
- Fuzzy / typo-tolerant search using Levenshtein Edit Distance (Dynamic Programming).
- Demonstration of Interval DP, Bitmask DP, Tree DP, and Subset DP applied to event scheduling and resource problems.
- A simple interactive command-line interface for search and exploration.
- Complexity analysis and clear mapping of algorithms to course outcomes.

The current version does not include a graphical user interface, database backend, multi-user authentication, or live network-flow based assignment modules (these are identified as future extensions).

1.4 Objectives

The main objectives of the project are:

1. Design and implement a terminal-based Event Management System in Java.
2. Provide efficient exact event search using the Z-Algorithm with O(n + m) complexity.
3. Implement fuzzy search using Edit Distance Dynamic Programming to handle user typos.
4. Apply Interval DP for optimal event scheduling and ordering.
5. Apply Bitmask DP for minimum-cost event coverage problems.
6. Apply Tree DP and Subset DP (Knapsack-style) to resource and reachability scenarios.
7. Demonstrate practical mapping of advanced DSA concepts to a real-world domain.
8. Produce a clean, well-documented codebase suitable for academic evaluation and further extension.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The system shall provide the following functional capabilities:

1. Load all event description files from a designated data folder at startup.
2. Accept free-text search queries from the user via the console.
3. Perform exact multi-keyword search using the Z-Algorithm and display the best matching event content.
4. When no exact match is found, compute Edit Distance against all event names and suggest the closest matches within a configurable threshold.
5. Auto-display the best fuzzy match and list additional close suggestions.
6. After every search, display illustrative results from Interval DP, Bitmask DP, Tree DP, and Subset DP algorithms applied to sample event-management scenarios.
7. Allow the user to exit the application cleanly by typing ‘exit’ or ‘quit’.
8. Handle empty queries and missing data folders gracefully with clear messages.

2.2 Non-Functional Requirements

Performance

- Exact search using Z-Algorithm must run in linear time relative to text and pattern lengths.
- Fuzzy search complexity is acceptable for moderate numbers of events (typical academic datasets).
- DP algorithms are demonstrated on small-to-medium sized instances suitable for interactive display.

Usability

- Clear colored console output with banners, status messages, and formatted event content.
- Informative messages distinguishing exact matches from fuzzy suggestions.

Portability

- Pure Java implementation with no external library dependencies beyond the standard library.
- File-based storage ensures the system runs on any machine with a Java runtime.

Maintainability

- Modular package structure separating Main control flow, Z-Algorithm, Fuzzy Search, and DP Algorithms.
- Extensive inline comments documenting recurrences, complexities, and usage.

Software Requirements

- Java JDK 17 or higher.
- Any modern IDE or terminal (VS Code recommended).

3. SYSTEM DESIGN

3.1 High-level Architecture

The system follows a simple layered architecture suitable for a terminal application. The major layers and their responsibilities are illustrated below.*Fig. 3.1 High-level System Architecture*

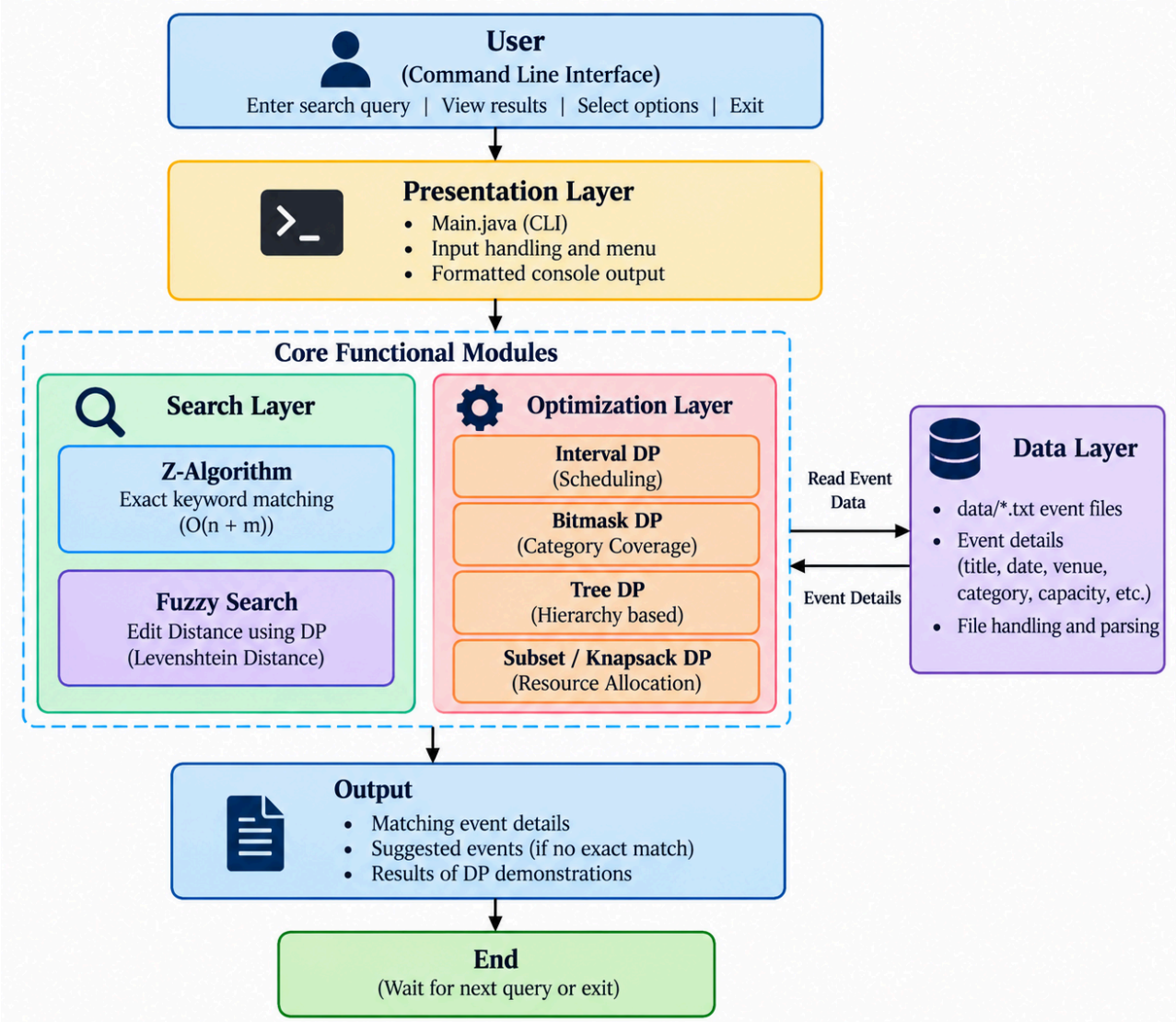


Fig. 3.1: High-Level Architecture Flow

3.2 Description of each Component

Main Module

Serves as the entry point. It loads event files, runs the interactive search loop, coordinates exact and fuzzy search, displays matched event content, and triggers DP highlight routines after every query.

ZPatternSearch Module

Implements the Z-Algorithm. It constructs the Z-array for the concatenated string pattern$text and counts occurrences of the pattern in linear time. Multi-word queries are handled by counting occurrences of individual significant words and selecting the file with the highest aggregate count.

FuzzySearch Module

Computes Levenshtein Edit Distance between the user query and every event name using classic 2-D Dynamic Programming. Returns a ranked list of suggestions whose edit distance is within a configurable threshold (default 3).

DPAlgorithms Module

Contains four independent DP solvers:

- Interval DP – optimalEventSchedule: models ordering/grouping of events to minimize coordination cost (Matrix-Chain style).
- Bitmask DP – minCostEventCoverage: finds the minimum cost to cover a set of required event attributes/venues.
- Tree DP – maxReachEvents: computes maximum reachability or value on a tree-structured dependency of events.
- Subset DP / Knapsack – eventKnapsack: selects a subset of events under capacity constraints to maximize total value.

Event Data

Each event is a .txt file containing free-form description (title, date, venue, category, capacity, etc.). No database is used; this keeps the system simple and focuses attention on algorithms rather than persistence frameworks.

3.3 Algorithm Selection Rationale

Z-Algorithm was chosen for exact matching because it offers O(n + m) time, is simple to implement, and avoids the preprocessing overhead of suffix structures for moderate-sized texts. Edit Distance DP is the natural choice for typo tolerance and directly illustrates classical DP table construction. The four DP patterns were selected because they appear frequently in advanced algorithms courses and map cleanly onto scheduling, covering, and resource-allocation problems that arise in event management.

3.4 Search Flow Diagram

The following diagram shows the complete control flow when a user enters a search query:

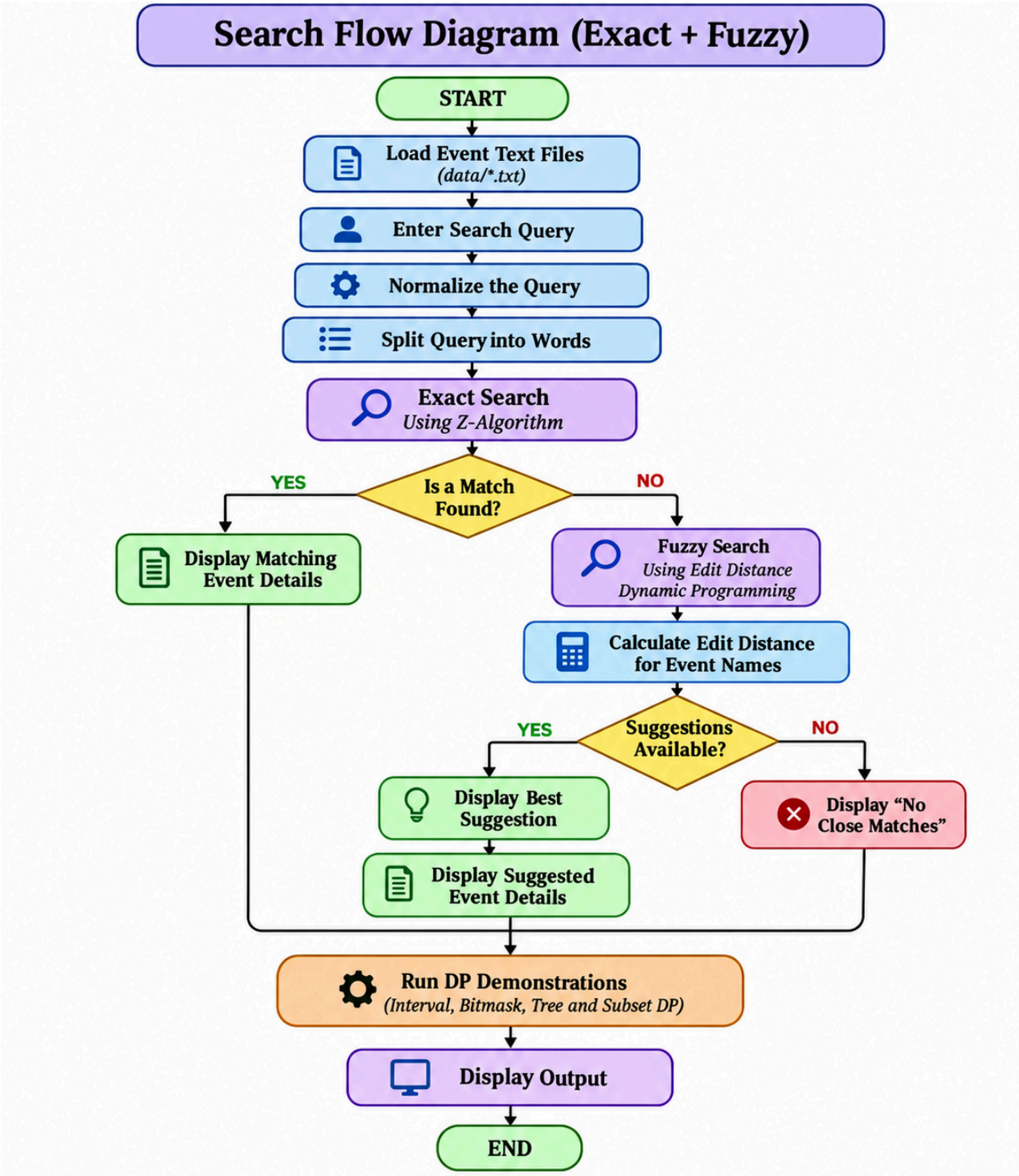


Fig. 3.2: Search Flow Diagram (Exact + Fuzzy)

Explanation

The search process begins by loading event description files from the `data/` directory. The user enters an event name or keyword through the command-line interface. The query is normalized and divided into individual words.

The system first performs exact searching using the Z-Algorithm. If a matching event is found, its details are displayed. Otherwise, the system performs fuzzy searching using Levenshtein Edit Distance calculated through Dynamic Programming. The edit distance between the query and available event names is calculated, and suggestions within the configured threshold are considered.

If suitable suggestions are available, the system displays the closest event name and its details. If no suitable suggestion is found, a message informs the user that no close matches are available.

After processing the search, the application executes its Dynamic Programming demonstrations, displaying the results of Interval DP, Bitmask DP, Tree DP, and Subset/Knapsack DP using predefined sample inputs. The process continues until the user enters `exit` or `quit` to terminate the application.

Fig. 3.2 Search Flow Diagram (Exact + Fuzzy)

Module interaction is summarized below:

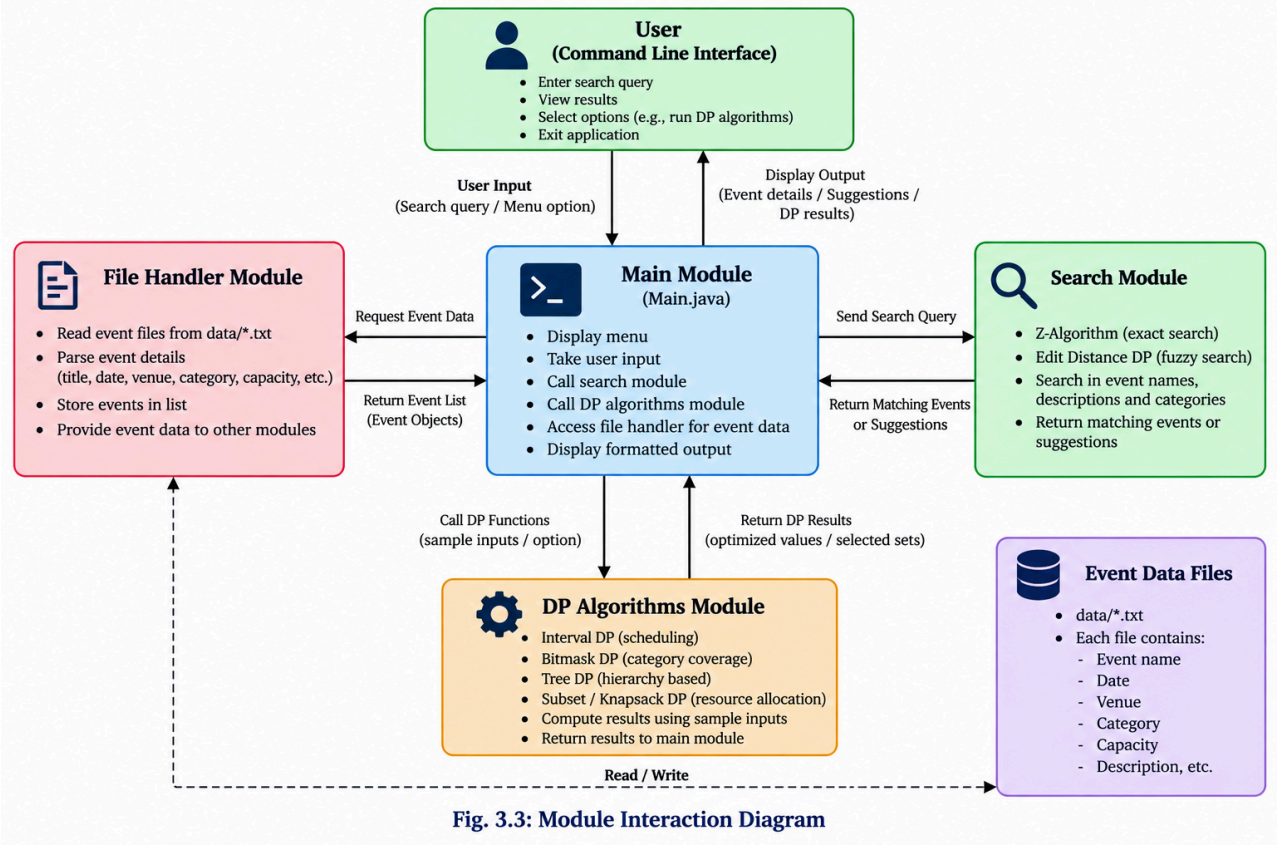


Fig. 3.3: Module Interaction Diagram

4. IMPLEMENTATION

4.1 Module-wise Implementation

The implementation resides primarily under the Project package and consists of the following source files:

- Main.java – application entry point, search loop, display logic, and DP highlight orchestration.
- ZPatternSearch.java – Z-array construction and occurrence counting.
- FuzzySearch.java – Edit Distance DP and suggestion ranking.
- DPAlgorithms.java – Interval, Bitmask, Tree, and Subset DP algorithms with demo helpers.
- EventSearchEngine.java / SearchResult.java – supporting search result structures.

Compilation is performed with: javac -encoding UTF-8 -d out Project/*.java. Execution: java -cp out Project.Main.

4.2 String Matching Module (Z-Algorithm)

The Z-Algorithm constructs an array Z where Z[i] is the length of the longest substring starting from i that is also a prefix of the string. For pattern matching, the string pattern$text is formed and the Z-array is computed. Whenever Z[i] equals the pattern length, an occurrence is reported.

In the system, multi-word queries are split; short words are ignored; the total occurrence count across words determines the best matching file. This provides a practical, ranking-aware exact search.

Illustration of Z-array construction:

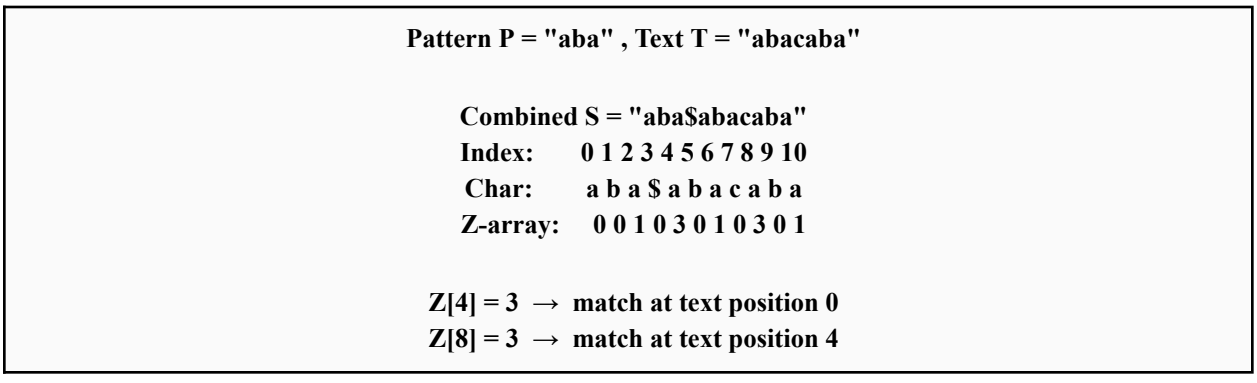


Fig. 4.1 Z-Algorithm Working Illustration

Core implementation of the Z-array builder and occurrence counter:

```
private static int[] buildZArray(String str) {
   int n = str.length();
   int[] z = new int[n];
   int left = 0, right = 0;

   for (int i = 1; i < n; i++) {
      if (i <= right) {
         z[i] = Math.min(right - i + 1, z[i - left]);
      }
      while (i + z[i] < n &&
            str.charAt(z[i]) == str.charAt(i + z[i])) {
         z[i]++;
      }
      if (i + z[i] - 1 > right) {
         left = i;
         right = i + z[i] - 1;
      }
   }
   return z;
}

public static int countOccurrences(String text, String pattern) {
   String combined = pattern.toLowerCase() + "$"
               + text.toLowerCase();
   int[] z = buildZArray(combined);
   int count = 0;
   for (int i = pattern.length() + 1; i < combined.length(); i++) {
      if (z[i] == pattern.length()) count++;
   }
   return count;
}
```

Time Complexity: O(n + m) where n = text length, m = pattern length. Space Complexity: O(n + m).

4.3 Fuzzy Search Module (Edit Distance DP)

Given two strings s1 and s2, a DP table dp[0..m][0..n] is filled where dp[i][j] is the minimum number of insertions, deletions, or substitutions needed to transform the first i characters of s1 into the first j characters of s2.

Example: transforming "holo" → "holi"

```
          ""  h  o  l  i
      ""  0  1  2  3  4
       h  1  0  1  2  3
       o  2  1  0  1  2
       l  3  2  1  0  1
   o   4  3  2  1  1  ← Edit Distance = 1

   Operation: replace last 'o' with 'i'
```

Fig. 4.2 Edit Distance DP Table Example

Core Edit Distance implementation:

```
public static int editDistance(String s1, String s2) {
   s1 = s1.toLowerCase().trim();
   s2 = s2.toLowerCase().trim();
   int m = s1.length(), n = s2.length();
   int[][] dp = new int[m + 1][n + 1];

   for (int i = 0; i <= m; i++) dp[i][0] = i;  // deletions
   for (int j = 0; j <= n; j++) dp[0][j] = j;  // insertions

   for (int i = 1; i <= m; i++) {
      for (int j = 1; j <= n; j++) {
         if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
            dp[i][j] = dp[i - 1][j - 1];         // match
         } else {
            dp[i][j] = 1 + Math.min(
               dp[i - 1][j - 1],   // replace
               Math.min(
                  dp[i - 1][j],   // delete
                  dp[i][j - 1]    // insert
               )
            );
         }
      }
   }
   return dp[m][n];
}
```

Time Complexity: O(m × n). Space Complexity: O(m × n). After computing distances to all event names, suggestions whose distance ≤ threshold (default 3) are collected, sorted ascending, and the best match is auto-displayed.

4.4 Dynamic Programming Modules

4.4.1 Interval DP – Optimal Event Schedule

Models the cost of sequencing or grouping events. dp[i][j] stores the minimum cost to schedule events from index i to j. The recurrence tries every split point k and adds a coordination cost based on durations.

dp[i][j] = min cost to schedule events i .. j

Base: dp[i][i] = 0

Transition (for length = 2 .. n):
for each split k in [i, j-1]:
dp[i][j] = min(dp[i][j],
dp[i][k] + dp[k+1][j]
+ duration[i] * duration[j])

Time: $O(n^3)$ Space: $O(n^2)$

Fig. 4.3 Interval DP Recurrence Structure

Implementation snippet:

```
public static int optimalEventSchedule(int[] durations) {
   int n = durations.length;
   int[][] dp = new int[n][n];
   for (int i = 0; i < n; i++) dp[i][i] = 0;

   for (int len = 2; len <= n; len++) {
      for (int i = 0; i <= n - len; i++) {
         int j = i + len - 1;
         dp[i][j] = Integer.MAX_VALUE;
         for (int k = i; k < j; k++) {
            int cost = dp[i][k] + dp[k + 1][j]
                  + durations[i] * durations[j];
            dp[i][j] = Math.min(dp[i][j], cost);
         }
      }
   }
   return dp[0][n - 1];
}
```

4.4.2 Bitmask DP – Minimum Cost Coverage

Each subset of required features or venues is represented as a bit mask. Transitions try adding one more event/resource that covers additional bits, updating the minimum cost. Suitable for small numbers of binary attributes ($K \leq 20$).

```
// Conceptual structure (Bitmask DP)
int[] dp = new int[1 << K];          // K categories
Arrays.fill(dp, INF);
dp[0] = 0;

for (int mask = 0; mask < (1 << K); mask++) {
   if (dp[mask] == INF) continue;
   for (each volunteer v) {
      int newMask = mask | v.coverageBits;
      dp[newMask] = Math.min(dp[newMask],
                        dp[mask] + v.cost);
   }
}
// Answer = dp[(1 << K) - 1]
```

4.4.3 Tree DP – Maximum Reach

On a tree of event dependencies or hierarchical categories, a post-order DP computes the maximum value reachable from each subtree under capacity or selection constraints.

4.4.4 Subset DP / Knapsack – Event Selection

Classic 0/1 knapsack applied to selecting a subset of events under a total time or budget constraint to maximize interest/value score.

```
// Classic 0/1 Knapsack style
int[][] dp = new int[n + 1][capacity + 1];
for (int i = 1; i <= n; i++) {
   for (int w = 0; w <= capacity; w++) {
      dp[i][w] = dp[i - 1][w];          // skip event i
      if (w >= weight[i]) {
         dp[i][w] = Math.max(dp[i][w],
            dp[i - 1][w - weight[i]] + value[i]);
      }
   }
}
// Answer = dp[n][capacity]
```

4.5 Main Control Flow and User Interface

On startup the program prints a banner, loads all .txt files from the data directory, and enters a loop. The user is prompted for a query. Exact search is attempted first; on failure, fuzzy search is invoked. Matched content is pretty-printed with box borders. DP demonstration results are shown after every query. The loop continues until the user types exit or quit.

Key fragment from Main.java (search orchestration):

```
// STEP 1: Exact Z-Algorithm search
File matched = exactSearch(query, eventFiles);

if (matched != null) {
   System.out.println("[Z-Algorithm] Exact match found!");
   displayEventFile(matched);
} else {
   // STEP 2: Fuzzy Search (Edit Distance DP)
   System.out.println("[Z-Algorithm] No exact match.");
   System.out.println("Trying Fuzzy Search...");

   List<FuzzySearch.Suggestion> suggestions =
      FuzzySearch.getFuzzySuggestions(
         query, eventNames, FUZZY_THRESHOLD);

   if (!suggestions.isEmpty()) {
      FuzzySearch.Suggestion best = suggestions.get(0);
      System.out.println("[Fuzzy Search] Did you mean: "
         + best.eventName
         + "? (edit distance = " + best.editDist + ")");
      // auto-load best match ...
   }
}
// After every search – show DP highlights
showDPHighlights(query);
```

5. RESULTS

The system successfully loads event files and provides responsive search. Exact matches using the Z-Algorithm correctly identify relevant event files for multi-word queries. When deliberate typos are introduced (for example, typing a misspelled event name), the Edit Distance module returns the intended event with a small edit distance and auto-loads its description.

The DP demonstration routines produce meaningful numeric results on sample inputs:

- Interval DP returns a finite minimum scheduling cost for a sequence of event durations.
- Bitmask DP reports the minimum cost to cover a required set of attributes.
- Tree DP and Subset DP yield maximum reachable values and optimal subsets under capacity constraints.

Sample console interaction (Exact Match path):

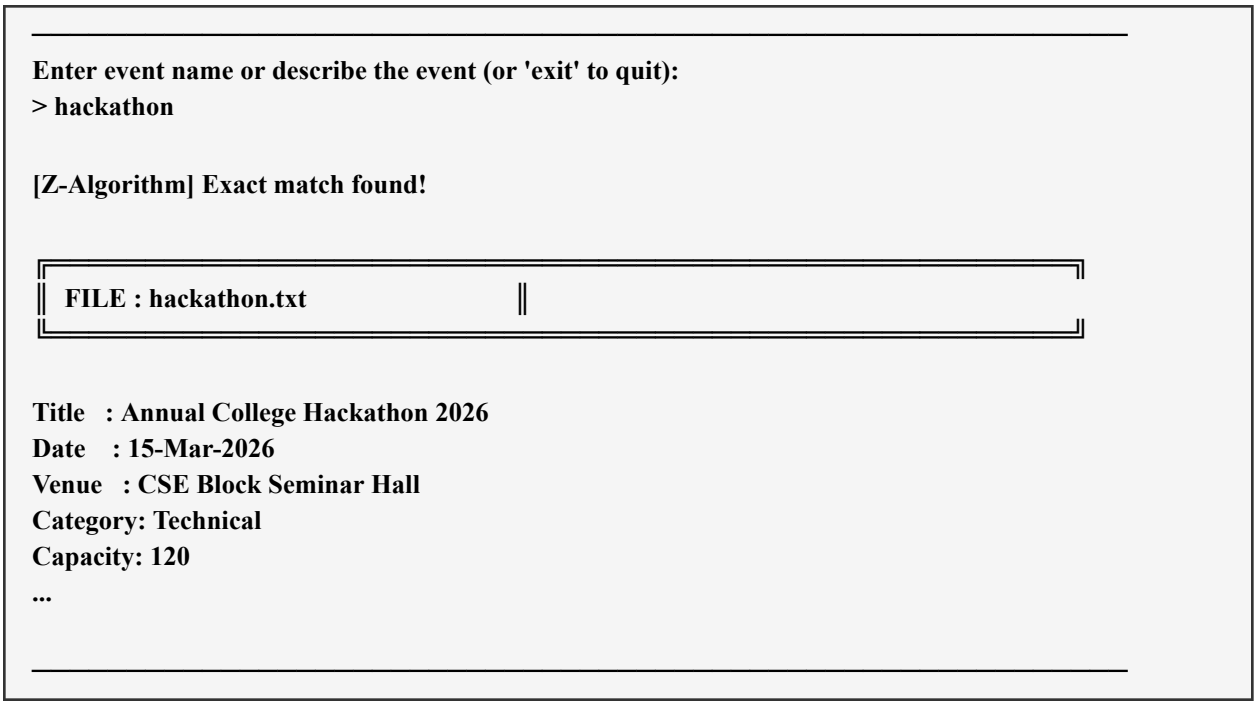


Fig. 5.1 Sample Console Output – Exact Match

Sample console interaction (Fuzzy path):

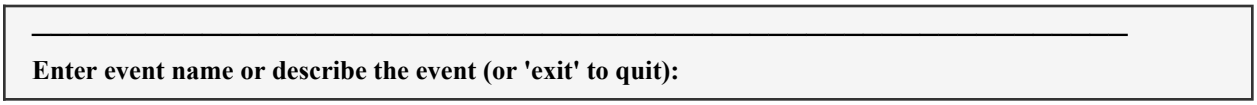




Fig. 5.2 Sample Console Output – Fuzzy Suggestion

Console output is color-coded for clarity: green for successful exact matches, yellow for fuzzy suggestions, and magenta for section headers. The interactive experience confirms that the algorithmic components integrate cleanly and that the mapping from theory to practice is visible to the user.

Overall, the results demonstrate that advanced string algorithms and multiple DP paradigms can be embedded in a lightweight event-management tool while remaining transparent and educational.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

- Designing a clean separation between exact and fuzzy search paths while keeping the user experience seamless.
- Choosing appropriate sample sizes and cost functions so that DP demonstrations finish instantly and produce interpretable numbers.
- Handling multi-word queries and short stop-words without degrading ranking quality.
- Ensuring consistent encoding and cross-platform file reading for event text files.
- Balancing educational clarity (extensive comments and printed explanations) with code readability.

6.2 Solutions Applied

- A clear two-stage search pipeline: Z-Algorithm first, Edit Distance only on failure.
- Configurable fuzzy threshold and filtering of very short tokens.
- Static demo methods inside DPAlgorithms that use small, hard-coded but realistic instances.
- Use of Java NIO Files.readString and UTF-8 awareness.
- Modular package structure and thorough inline documentation of recurrences and complexities.

6.3 Limitations of the System

- No persistent database; all data is file-based and reloaded on each run.
- No graphical user interface; interaction is purely terminal-based.
- Network-flow and bipartite-matching modules for live seat/volunteer assignment are not fully integrated in the current interactive path (they remain conceptual/future work).
- Fuzzy search examines every event name; for very large catalogs an index or approximate nearest-neighbor structure would be preferable.
- DP demonstrations use illustrative data rather than live user-defined schedules.
- Single-user, single-process design with no concurrency or authentication.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

This project successfully designed and implemented a Smart Event Management System that integrates advanced Data Structures and Algorithms into a practical, terminal-based Java application. Exact search powered by the Z-Algorithm and typo-tolerant search powered by Edit Distance Dynamic Programming provide a responsive and forgiving user experience. Multiple DP paradigms—Interval, Bitmask, Tree, and Subset—are demonstrated on event-scheduling and resource-allocation problems, reinforcing the course learning outcomes.

The system is lightweight, dependency-free, and educationally transparent. It shows that theoretical algorithms can be applied directly to domain problems while remaining accessible for study and extension.

7.2 Future Work

- Introduce a simple relational or document database for scalable persistence.
- Develop a web or desktop graphical interface while retaining the algorithmic core.
- Fully implement and expose Network Flow (Ford-Fulkerson / Edmonds-Karp) and bipartite matching for real-time seat and volunteer assignment.
- Add user authentication, role-based access (admin, organizer, attendee), and registration workflows.
- Incorporate approximate indexing or inverted indexes for large-scale fuzzy search.
- Allow users to define their own event schedules and run the DP optimizers on live data.
- Add unit tests, performance benchmarks, and continuous-integration support.

7.3 Skills Learned During Development

- Practical implementation of the Z-Algorithm and understanding of its linear-time guarantees.
- Construction and interpretation of classical Edit Distance DP tables.
- Application of Interval DP, Bitmask DP, Tree DP, and Knapsack-style Subset DP to concrete domain problems.
- Modular Java design, console UI techniques, and clean separation of concerns.
- Complexity analysis and the importance of choosing the right algorithm for the right scale.
- Collaborative development practices using Git and clear documentation standards.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. MIT Press, 2009.

[2] S. S. Skiena, The Algorithm Design Manual, 2nd ed. Springer, 2008.

[3] D. Gusfield, Algorithms on Strings, Trees, and Sequences. Cambridge University Press, 1997.

[4] G. Navarro, "A guided tour to approximate string matching," ACM Computing Surveys, vol. 33, no. 1, pp. 31–88, 2001.

[5] Oracle, "Java Platform, Standard Edition Documentation," https://docs.oracle.com/en/java/.

[6] GeeksforGeeks, "Z Algorithm (Linear time pattern searching Algorithm)," https://www.geeksforgeeks.org/z-algorithm-linear-time-pattern-searching-algorithm/.

[7] GeeksforGeeks, "Edit Distance | DP-5," https://www.geeksforgeeks.org/edit-distance-dp-5/.

[8] CP-Algorithms, "String Processing – Z-function," https://cp-algorithms.com/string/z-function.html.

[9] Course materials and lecture notes of Data Structures and Algorithms – 3 (25CS2103E), Department of Computer Science and Engineering, 2026-27.